

SALORIE

Rapport Technique Complet

Application mobile de suivi calorique avec IA
Expo SDK 52 • React Native 0.76 • Firebase • Clerk • Gemini 2.5

Date : 22 avril 2026
Auteur : Equipe Salorie

Table des matieres

- 1. Architecture generale
- 2. Stack technique et dependances
- 3. Structure des fichiers - explication detaillee
- 4. Routes et navigation (expo-router)
- 5. Modeles Firestore (collections et documents)
- 6. Fonctionnalités (features)
- 7. Scenarios de test (unitaires, integration, E2E, edge cases)
- 8. Sécurité, performance et déploiement

1. Architecture generale

Salorie est une application mobile multiplateforme (iOS + Android) construite avec Expo SDK 52 et React Native 0.76.9 en mode *new architecture / bridgeless*. L'architecture suit le pattern d'une application mobile moderne a trois couches : presentation (ecrans et composants React Native), couche domaine (contextes React et services dans **lib/**) et couche donnees (Firebase Firestore pour la persistance, AsyncStorage pour le cache local, SecureStore pour les tokens, et APIs externes : Gemini 2.5 Flash, FatSecret OAuth2, Clerk Auth, RevenueCat).

Diagramme en couches

UI LAYER (app/*.tsx, components/*.tsx)	
Expo Router file-based routes + shared components	
DOMAIN LAYER (lib/*.tsx)	
LoggingContext - ThemeContext - i18n - AiModel - etc.	
DATA LAYER	
Firebase Firestore AsyncStorage SecureStore	
EXTERNAL APIs	
Clerk Gemini 2.5 FatSecret RevenueCat Unsplash	

Flux de donnees principal

- **Auth** : Clerk gere les sessions (email/password, Google OAuth). Le token est stocke via expo-secure-store.
- **Onboarding** : les reponses sont ecrites dans Firestore via lib/firebase.ts → saveUserToFirestore().
- **Logging alimentaire** : scan photo (expo-camera CameraView inline) → Gemini Vision 2.5 Flash → JSON macros → Firestore **nutritionLogs** → triggerRefresh() → UI rafraichie.
- **Logging exercice** : formule MET + raffinement Gemini → Firestore nutritionLogs type='activity' → jauge calories mise a jour.
- **Cache** : LoggingContext garde les logs du jour en memoire, AsyncStorage persiste entre sessions (LocalDataStore).
- **i18n** : useTranslation() hook ; 3 langues (EN/FR/AR) avec support RTL pour l'arabe.
- **Theme** : ThemeContext resout 'system' / 'light' / 'dark' et expose la palette.

2. Stack technique et dependances

Domaine	Librairie	Version	Role
Framework	Expo SDK	52.0.49	Build tooling + OTA
Runtime	React Native	0.76.9	Bridgeless (new architecture)
UI	React	18.3.1	Composants fonctionnels + hooks
Routing	expo-router	4.0	File-based navigation
Auth	@clerk/clerk-expo	2.19	Session + OAuth + JWT
DB	firebase	12.12	Firestore
Cache local	AsyncStorage	1.23	Persistance hors-ligne
Secrets	expo-secure-store	14.0	Stockage securise tokens
IA Vision	@google/generative-ai	0.24	Gemini 2.5 Flash multimodal
Camera	expo-camera	16.0	CameraView inline (pas d'Intent)
Animations	react-native-reanimated	3.16	Transitions UI fluides
Icones	lucide-react-native	1.8	Icones vectorielles
Graphiques	react-native-chart-kit	6.12	Analytics
Listes perf.	@shopify/flash-list	1.7	Scroll virtualise
IAP	react-native-purchases	10.0	RevenueCat premium
Notifications	expo-notifications	0.29	Rappels repas/eau

3. Structure des fichiers - explication detaillee

3.1 Dossier app/ (routes expo-router)

Fichier	Description
_layout.tsx	Layout racine. Fournit ClerkProvider, ThemeProvider, i18nProvider, LoggingProvider. Monte le Stack de navigation, gere le splash screen (expo-splash-screen) et les polices custom.
index.tsx	Point d'entree. Redirige vers (auth)/sign-in si non connecte, sinon vers (onboarding) si profil incomplet, sinon vers (tabs).
welcome.tsx	Ecran d'accueil marketing avant sign-in. Presente les features premium.
(auth)/_layout.tsx	Stack nav pour sign-in et sign-up.
(auth)/sign-in.tsx	Connexion Clerk email + Google OAuth. Stocke le token en SecureStore.
(auth)/sign-up.tsx	Inscription Clerk avec verification email (OTP).
oauth-callback.tsx	Gere le retour d'un flux OAuth externe (Google).
(onboarding)/_layout.tsx	Stack nav pour l'onboarding en plusieurs etapes.
(onboarding)/index.tsx	Questionnaire multi-etapes (genre, age, taille, poids, objectif, niveau d'activite). Utilise react-native-ruler-picker.
(onboarding)/results.tsx	Affiche l'objectif calorique calcule (Mifflin-St Jeor x facteur activite) et enregistre dans Firestore.
(tabs)/_layout.tsx	Bottom tabs (Home / Analytics / Profile) + FAB central pour ActionMenu.
(tabs)/index.tsx	Home : WeekCalendar, RemainingCaloriesCard, WaterIntakeCard, ActivityList.
(tabs)/analytics.tsx	Graphiques line/bar : evolution hebdo/mensuelle des calories, macros, eau, exercice.
(tabs)/profile.tsx	Profil, acces preferences/terms/privacy/notifications/feature-requests, langue, theme, sign-out.
scan-camera.tsx	CameraView expo-camera INLINE (evite l'Intent Android qui tue l'activity RN en Expo Go). Capture photo -> FileSystem URI -> push scan-analysis.
scan-analysis.tsx	Affiche la photo, lance Gemini Vision 2.5 Flash avec prompt multilingue retournant {name, quantity, unit, description, calories, protein, carbs, fat}. Barre d'animation pendant analyse. Header theme + i18n. Gestion URI '%25' pour Expo Go.
log-food-details.tsx	Resume editable avant sauvegarde : nom, quantite, unite, macros, image. Bouton Log -> addNutritionLog(type='meal') -> triggerRefresh() -> retour home.
log-exercise.tsx	Hub d'exercice a 3 options : Run, Lifting, Manual. Header + theme + i18n + RTL.
workout-details.tsx	Ecran commun pour cardio et lifting. Selection du sous-type (running/walking/cycling/swimming/hiking/rowing ou bench_press/squat/deadlift/shoulder_press/pullup/bicep_curl) via chips images Unsplash. Intensite low/med/high. Duree chips + custom. Calcul MET (MET*3.5*poids/200*duree) puis raffinement Gemini text -> moyenne.
workout-result.tsx	'Your workout burned X kcal'. Image hero, intensite badge, Firebase save (type='activity'), triggerRefresh().
log-manual.tsx	Saisie manuelle calories + macros. Photo optionnelle. Update cache (LoggingContext) + DB Firestore simultanes.

Fichier	Description
add-water.tsx	Compteur eau ml (+-125 ml par verre). Visualisation empty/half/full glass. Firestore type='water'.
food-database.tsx	Recherche FatSecret OAuth2. Query -> searchFood() -> liste resultats -> selection -> log-food-details.
update-weight.tsx	Maj poids avec ruler-picker. Met a jour Firestore users doc.
personal-details.tsx	Edition du profil (nom, age, etc.).
preferences.tsx	Langue, theme, unites, objectifs caloriques, notifications.
notifications.tsx	Historique notifications + reglages (expo-notifications).
feature-requests.tsx	Formulaire de demande de features (Firestore collection featureRequests).
terms.tsx	Conditions d'utilisation (texte statique multilingue).
privacy.tsx	Politique de confidentialite.
+not-found.tsx	Fallback 404.

3.2 Dossier components/

Composant	Description
ScreenTopBar.tsx	Barre superieure partagee : logo AppBrand + pill langue + toggle theme + cloche notifications (optionnelle).
AppBrand.tsx	Logo + nom de l'app.
HomeHeader.tsx	Header specifique a Home (salutation + avatar + date).
WeekCalendar.tsx	Selecteur de jour (lun-dim) avec scroll, couple au selectedDate de LoggingContext.
RemainingCaloriesCard.tsx	Jauge calories restantes (HalfProgress). Icone crayon ouvre modal DailyTargets (useTheme + useTranslation).
HalfProgress.tsx	Arc SVG semi-circulaire (react-native-svg) affichant % consomme/restant.
CaloriesCard.tsx	Carte macros (proteines/glucides/lipides) avec barres de progression.
WaterIntakeCard.tsx	Carte eau bue vs objectif + bouton quick-add.
ActivityList.tsx	Liste des logs du jour (meals, activities, water) trie par timestamp.
ActionMenu.tsx	Modal bottom-sheet FAB : 4 cartes (exercise, water, food-database, scan-food). i18n via t('menu.*').
LogModal.tsx	Modal legacy de saisie rapide log.
ScreenBackground.tsx	Wrapper avec fond theme + gradient optionnel.

3.3 Dossier lib/

Fichier	Description
firebase.ts	Initialise Firebase. Helpers : saveUserToFirestore(), getUserFromFirestore(), updateUserField(), addNutritionLog(), getNutritionLogs(date), deleteNutritionLog(), updateWaterTarget().
LoggingContext.tsx	Contexte global : selectedDate, logs[], triggerRefresh(), showLogModal, isActionMenuVisible, scanImageBase64. Fetch auto Firestore a chaque changement de selectedDate.
ThemeContext.tsx	Mode light/dark/system. Resout 'system' via Appearance. Persiste dans AsyncStorage. Expose {mode, resolved, colors, setMode}.
i18n.tsx	3 langues EN/FR/AR. Dictionnaire statique + useTranslation() -> {language, t, setLanguage, isRTL}. Persiste dans AsyncStorage.
AiModel.ts	Wrapper GoogleGenerativeAI. Fonctions : analyzeFoodImage(base64), refineCaloriesEstimate(prompt).
fatsecret.ts	OAuth2 FatSecret. getAccessToken() avec cache TTL, searchFood(query) -> foods.search. Retry 401 avec refresh.
translator.ts	Helper pour traduire les noms d'aliments renvoyes par FatSecret.
InsightsService.ts	Analytics : moyennes 7j/30j, tendances, ecart vs objectif.
NotificationService.ts	Expo-notifications : registerForPushNotifications, scheduleDailyReminder (repas/eau).
PurchasesService.ts	RevenueCat : configure, getOfferings, purchasePackage, checkPremium.
LocalDataStore.ts	Cache AsyncStorage structure par date + utilitaires colorLog(COLOR,label,obj) et explain(text) pour debug.

3.4 Autres dossiers

- **constants/Colors.ts** : palette globale (primary #22C55E, primaryLight, gray[50..900], white).
- **hooks/** : hooks partagés (ex: useColorScheme wrapper).
- **assets/images/** : logos, splash, empty_glass, half_glass, full_glass, icones.
- **android/** : projet natif Android (gradle, AndroidManifest) pour dev-build.
- **app.json** : configuration Expo (permissions CAMERA/NOTIFICATIONS, plugins, extra: firebase/clerk/gemini/fatsecret).
- **eas.json** : profils de build EAS (development/preview/production).
- **tsconfig.json** : strict TS + path aliases.

4. Routes et navigation

Expo-router utilise une convention file-based : chaque fichier .tsx dans **app/** devient une route ; les dossiers (**nom**) sont des groupes (pas de segment d'URL) ; **_layout.tsx** wrap les enfants.

Route	Fichier	Guard	Description
/	app/index.tsx	Public	Redirection conditionnelle.
/welcome	app/welcome.tsx	Public	Landing marketing.
/auth/sign-in	(auth)/sign-in.tsx	Public	Clerk sign-in.
/auth/sign-up	(auth)/sign-up.tsx	Public	Clerk sign-up + OTP.
/oauth-callback	oauth-callback.tsx	Public	Retour OAuth Google.
/onboarding	(onboarding)/index.tsx	Auth	Questionnaire.
/onboarding/results	(onboarding)/results.tsx	Auth	Calcul objectif + save.
/tabs	(tabs)/index.tsx	Auth+Onboarded	Home.
/tabs/analytics	(tabs)/analytics.tsx	Auth	Graphiques.
/tabs/profile	(tabs)/profile.tsx	Auth	Profil.
/scan-camera	scan-camera.tsx	Auth	Camera inline.
/scan-analysis	scan-analysis.tsx	Auth	Gemini Vision.
/log-food-details	log-food-details.tsx	Auth	Validation log repas.
/log-exercise	log-exercise.tsx	Auth	Hub exercice.
/workout-details	workout-details.tsx	Auth	Selection type+intensite+duree.
/workout-result	workout-result.tsx	Auth	Resume calories brulees.
/log-manual	log-manual.tsx	Auth	Saisie manuelle.
/add-water	add-water.tsx	Auth	Ajout eau.
/food-database	food-database.tsx	Auth	FatSecret search.
/update-weight	update-weight.tsx	Auth	Maj poids.
/personal-details	personal-details.tsx	Auth	Edition profil.
/preferences	preferences.tsx	Auth	Preferences.
/notifications	notifications.tsx	Auth	Notifications.
/feature-requests	feature-requests.tsx	Auth	Demandes features.
/terms	terms.tsx	Public	CGU.
/privacy	privacy.tsx	Public	Privacy.
/+not-found	+not-found.tsx	Public	404.

Params typiques

- **/scan-analysis** : {imageUri: string}
- **/log-food-details** : {name, quantity, unit, description, calories, protein, carbs, fat, imageUri}

- **/workout-details** : {type: 'run' | 'lifting'}
- **/workout-result** : {calories, name, duration, type, activityId, image, intensity}

5. Modeles de base de donnees Firestore

Chaque collection est detaillee ci-dessous avec la structure de document, les index requis et les regles de securite.

5.1 Collection users

Document ID : email de l'utilisateur (identifiant Clerk primaire).

```
{
  id: string,                // clerkUserId
  email: string,
  firstName: string,
  lastName: string,
  gender: "male" | "female",
  age: number,
  heightCm: number,
  weightKg: number,
  goal: "lose" | "maintain" | "gain",
  activityLevel: "sedentary" | "light" | "moderate" | "active" | "very_active",
  calorieGoal: number,      // kcal/jour, Mifflin-St Jeor x facteur
  proteinGoalG: number,
  carbsGoalG: number,
  fatGoalG: number,
  waterGoalMl: number,      // default 2500
  language: "en" | "fr" | "ar",
  themeMode: "light" | "dark" | "system",
  premium: boolean,
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

5.2 Collection nutritionLogs

Document ID : auto-generé. Une entrée par repas, activité ou verre d'eau.

```
{
  id: string,                // auto
  userId: string,            // email
  type: "meal" | "activity" | "water",
  name: string,              // ex "Pizza Margherita", "Running", "Water Intake"
  date: string,              // YYYY-MM-DD (local timezone)
  timestamp: Timestamp,

  // MEAL uniquement
  quantity?: number,
  unit?: "g" | "ml",
  description?: string,
  imageUrl?: string,
  calories: number,         // kcal (ml pour water)
  protein: number,
  carbs: number,
  fat: number,

  // ACTIVITY uniquement
  activityId?: string,      // "running" | "bench_press" | ...
  intensity?: "low" | "medium" | "high",
  durationMin?: number,
  met?: number
}
```

Index composites requis

- (userId ASC, date ASC, timestamp DESC) - liste logs du jour.

- (userId ASC, type ASC, timestamp DESC) - filtrage analytics par type.

5.3 Collection featureRequests

```
{
  id: string,
  userId: string,
  title: string,
  description: string,
  category: "ui" | "ai" | "integration" | "other",
  status: "open" | "planned" | "done" | "rejected",
  votes: number,
  createdAt: Timestamp
}
```

5.4 Sub-collection users/{email}/notifications

```
{
  id: string,
  title: string,
  body: string,
  type: "meal_reminder" | "water_reminder" | "streak" | "system",
  read: boolean,
  createdAt: Timestamp
}
```

5.5 Regles de securite Firestore

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /users/{email} {
      allow read, write: if request.auth != null
        && request.auth.token.email == email;
      match /notifications/{nid} {
        allow read, write: if request.auth.token.email == email;
      }
    }
    match /nutritionLogs/{id} {
      allow read, write: if request.auth != null
        && request.auth.token.email == resource.data.userId;
      allow create: if request.auth.token.email
        == request.resource.data.userId;
    }
    match /featureRequests/{id} {
      allow read, create: if request.auth != null;
    }
  }
}
```

6. Fonctionnalités

Feature	Description
Authentification	Clerk (email+password, Google OAuth, verification OTP). Session persistee via expo-secure-store. Deconnexion depuis l'ecran profil.
Onboarding personnalise	Questionnaire multi-ecrans (genre, age, taille, poids, objectif, niveau d'activite). Calcul automatique du BMR (Mifflin-St Jeor) x facteur activite = calorieGoal. Macros : 30% proteines, 40% glucides, 30% lipides par default.
Home dashboard	WeekCalendar pour naviguer jour par jour. Jauge calories restantes (HalfProgress SVG). Carte macros. Carte eau. Liste d'activite chronologique du jour.
Scan alimentaire IA	Photo via expo-camera CameraView INLINE (evite l'Intent Android qui tue l'activity RN en Expo Go). Gemini 2.5 Flash multimodal recoit l'image + prompt multilingue. Retourne JSON {name, quantity, unit g/ml, description, calories, protein, carbs, fat}. Ecran log-food-details pre-rempli permet edition avant sauvegarde.
Recherche alimentaire	base FatSecret OAuth2 (client credentials). Cache du token (TTL 24h). Retry 401 avec refresh automatique. Resultats paginals (max 5).
Saisie manuelle	log-manual.tsx : champs calories + macros + nom + photo optionnelle. Met a jour cache LoggingContext et Firestore simultanement.
Eau quotidienne	add-water.tsx : increments 125 ml (demi-verre) jusqu'a 1000 ml. Animation glasses empty/half/full. Log type='water' avec calories=ml (convention).
Log exercice cardio	workout-details avec type='run'. 6 activites : running, walking, cycling, swimming, hiking, rowing. Formule MET : $(MET \times 3.5 \times \text{poids} / 200) \times \text{duree_min}$. Raffinement Gemini text puis moyenne. Intensite low/med/high change le MET.
Log exercice musculation	workout-details avec type='lifting'. 6 exercices : bench_press, squat, deadlift, shoulder_press, pullup, bicep_curl. MET ajuste par intensite + duree. Images Unsplash.
Workout result	Affiche 'Your workout burned X kcal'. Image hero, intensite badge. Bouton Log Workout -> addNutritionLog(type='activity') -> triggerRefresh().
Analytics	(tabs)/analytics.tsx : graphiques line/bar react-native-chart-kit. Tendances 7j/30j calories, macros, eau, exercice. InsightsService calcule moyennes et ecart vs objectif.
Profil et preferences	Edition poids/taille, preferences langue (EN/FR/AR), theme (light/dark/system), unites, objectifs caloriques personnalises, notifications on/off.
Internationalisation	3 langues avec RTL pour arabe. useTranslation() dans chaque ecran. Detection automatique de la langue systeme au premier lancement.
Theme clair/sombre	ThemeContext resout 'system' via Appearance.getColorScheme(). Toggle rapide dans ScreenTopBar. Palette coherente bg/textPrimary/textMuted/cardBg/cardBorder.
Notifications push	expo-notifications. Rappels quotidiens configurables : petit-dej, dejeuner, diner, eau. Page historique.
Abonnement Premium	RevenueCat. Paywall natif (react-native-purchases-ui). Features premium : scans illimites, export donnees, themes exclusifs.
Feature requests	Formulaire in-app, ecrit dans Firestore featureRequests avec vote.
Hors-ligne	AsyncStorage cache les logs recents. La jauge calories reste fonctionnelle sans connexion ; sync automatique au retour en ligne.

7. Scenarios de test

Couverture ciblée : unitaires (Jest), integration (React Native Testing Library), E2E (Detox), et tests manuels (sanity).

7.1 Tests unitaires (lib/)

- firebase.ts - addNutritionLog() écrit un doc avec userId, date, type corrects.
- firebase.ts - getUserFromFirestore() retourne null si doc inexistant.
- fatsecret.ts - getAccessToken() cache le token pendant TTL.
- fatsecret.ts - searchFood() déclenche un refresh sur 401 puis retry.
- AiModel.ts - analyzeFoodImage() gère un JSON malformé et retourne une erreur typée.
- i18n.tsx - t(key) retourne la clé elle-même si la langue n'a pas la traduction.
- i18n.tsx - isRTL === true pour arabe, false pour en/fr.
- ThemeContext - resolved === 'dark' quand mode='system' et Appearance='dark'.
- LoggingContext - triggerRefresh() incrémente le compteur qui re-exécute useEffect.
- LocalDataStore - colorLog n'émet pas en production (NODE_ENV).
- InsightsService - moyenne 7j correcte malgré jours manquants.
- BMR Mifflin-St Jeor : homme 30 ans 80 kg 180 cm -> ~1780 kcal.

7.2 Tests d'integration (ecrans)

- sign-in : mauvais password -> alert 'Invalid credentials' affichée.
- sign-up : email invalide -> bouton désactivé.
- onboarding : progression 0-100% en 6 étapes, bouton back fonctionnel.
- onboarding/results : calorieGoal affiche correspond au calcul attendu (homme, 25 ans, 75 kg, 175 cm, maintenir, modéré).
- Home : changer selectedDate via WeekCalendar recharge les logs.
- scan-camera : permission camera refusée -> Alert + retour.
- scan-analysis : Gemini timeout -> message d'erreur + bouton retry.
- scan-analysis : fichier image s'affiche malgré le '%' littéral dans le chemin Expo Go (re-encodage '%25').
- log-food-details : bouton Log désactivé si calories === 0.
- log-food-details : sauvegarde -> retour home -> jauge calories mise à jour.
- log-exercise : les 3 cartes sont traduites dans la langue active.
- workout-details (run) : chaque activité affiche la bonne image Unsplash.
- workout-details : MET x 3.5 x 80 / 200 x 30 ≈ 126 kcal pour running 80kg 30min mod (MET 8).
- workout-result : image hero = param.image reçu.
- add-water : + 8 fois -> 1000 ml max (bouton + désactivé).
- add-water : Log -> Firestore doc crée type='water' calories=ml.
- food-database : recherche 'pizza' renvoie au moins 3 résultats.

- food-database : recherche vide desactive le bouton search.
- ActionMenu : les 4 titres sont traduits (EN/FR/AR).
- RemainingCaloriesCard : pencil -> modal Daily Targets avec labels traduits.
- RTL arabe : fleche back inversee (scaleX: -1), textAlign right.
- Dark mode : bg = #0B0F14 pour tous les ecrans refactores.

7.3 Tests E2E (Detox)

- Nouveau user : sign-up -> OTP -> onboarding -> home (premiere ouverture).
- User existant : sign-in -> home directement.
- Scan flow complet : FAB -> Scan Food -> Take Photo -> capture -> Gemini -> edit -> Log -> verif jauge +X kcal.
- Workout flow : FAB -> Log Exercise -> Run -> running -> med -> 30 min -> Continue -> workout-result -> Log -> verif activite dans ActivityList.
- Water flow : FAB -> Add Water -> +125x4 -> Log -> jauge water +500 ml.
- Food DB : FAB -> Food Database -> 'pizza' -> selection -> log-food-details -> Log.
- Manual : FAB -> Log Exercise -> Manual -> saisie -> Save -> verif cache + DB.
- Changement langue EN-FR-AR : tous les textes changent, layout RTL en AR.
- Changement theme light-dark-system : couleurs de fond changent.
- Sign-out -> redirection sign-in -> impossible d'acceder (tabs) directement.
- Mode avion : logs locaux s'affichent ; nouveau log mis en file -> sync au retour en ligne.

7.4 Edge cases et cas limites

- Photo 0 byte -> Gemini renvoie erreur -> Alert.
- Nom d'aliment vide apres edition -> bouton Log desactive.
- Quantite negative saisie -> valeur clampée a 0.
- Date future selectionnee dans WeekCalendar -> lecture seule, bouton Log desactive.
- User supprime cote Clerk mais encore en session -> 401 Firestore -> deconnexion forcee.
- Token FatSecret expire pendant une recherche -> retry transparent.
- Expo Go tue l'app pendant la camera -> pending_scan_v1 dans AsyncStorage -> relance scan-analysis au reboot.
- Gemini renvoie JSON avec backticks markdown -> parser tolerant.
- Utilisateur depasse sa limite calorique -> jauge passe rouge, pas de blocage.
- 4G lente : spinner > 10s -> timeout + bouton cancel.
- Mode RTL : TextInput curseur a droite, padding inverse.
- Multiple instances app ouvertes (tablette split-screen) -> LoggingContext coherent via AsyncStorage.
- Theme systeme change en live (iOS Auto dark a 18h) -> UI re-render.
- Migration schema users (ajout champ) : backfill defensif dans getUserFromFirestore.
- Image URI avec espaces ou caracteres non-ASCII -> encodage correct.

- Timezone : log a 23:55 vs minuit -> date YYYY-MM-DD locale, pas UTC.
- Deux devices meme compte : triggerRefresh recu via onSnapshot Firestore (si active).

7.5 Tests de performance

- Liste ActivityList de 500 items -> scroll fluide (FlashList).
- Gemini Vision < 5s pour photo 200 KB.
- Recherche FatSecret < 1.5s.
- Ouverture Home depuis cold start < 2s.
- Bundle size < 40 MB (EAS production).
- Consommation RAM camera < 300 MB (sinon kill Android).

7.6 Tests de securite

- Regles Firestore : utilisateur A ne peut pas lire les logs de B (test avec deux comptes).
- Cles API (Gemini, FatSecret) non exposees dans le bundle JS (Constants.expoConfig.extra).
- Tokens Clerk stockes en SecureStore (pas AsyncStorage).
- Pas de logs avec donnees sensibles (email complet, token) en production.
- Input sanitization : injection FatSecret via query speciale.
- Navigation guards : acces direct /(tabs) sans session -> redirect sign-in.

8. Securite, performance et deploiement

Securite

- Authentification Clerk avec JWT court (15 min) + refresh token en SecureStore.
- Regles Firestore strictes par userId (email Clerk).
- Aucune cle API en clair dans le repo : toutes dans app.json -> extra (EAS Secrets recommande en prod).
- HTTPS uniquement pour toutes les API (Firestore, Gemini, FatSecret).
- Permissions Android declarees : CAMERA, READ_MEDIA_IMAGES, POST_NOTIFICATIONS, INTERNET.
- iOS : NSCameraUsageDescription, NSPhotoLibraryUsageDescription.

Performance

- expo-camera CameraView INLINE evite l'Intent Android et donc le kill d'activity RN en Expo Go.
- Images Gemini compressees a quality 0.3 (~200 KB) pour reduire RAM et latence.
- FlashList pour listes longues (virtualized).
- Reanimated 3 en UI thread (new architecture bridgeless).
- AsyncStorage cache pour affichage instant offline.
- Token FatSecret mis en cache pour economiser un round-trip OAuth.

Deploiement

- EAS Build profils : development (dev-client), preview (APK testeurs), production (AAB / IPA).
- OTA via EAS Update pour patches JS instantanes.
- CI recommande : GitHub Actions -> eas build --non-interactive --profile production sur tag.
- Stores : Google Play (Android), App Store (iOS, TestFlight pour beta).
- Monitoring : Expo Dashboard + Sentry (a integrer).

Conclusion

Salorie combine une UX mobile soignee (animations Reanimated, theme clair/sombre, 3 langues avec RTL) avec une pile IA moderne (Gemini 2.5 Flash pour la reconnaissance nutritionnelle et le raffinement des estimations sportives) et une infrastructure cloud eprouvee (Clerk + Firestore). Ce rapport couvre tous les aspects techniques necessaires pour onboarder un nouveau developpeur, auditer la securite, ou etendre l'application avec de nouvelles features.